

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018



A PROJECT REPORT

ON

“Custom AWS Network Setup using VPC, Subnet, Internet Gateway, Route Table, and EC2”

*Submitted in partial fulfillment of the requirement for award of degree
of*

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

by

ROHIT KUMAR 1EP23CS109

Under the guidance of

Ajey Kumar



2025-2026

1. Objective

The goal of this project is to design and set up a custom Virtual Private Cloud (VPC) on AWS with a public subnet. The subnet is connected to the internet using an Internet Gateway and a properly configured route table, and an EC2 instance is launched inside it to confirm that it can be reached over the internet.

2. AWS Services Used

- VPC (Virtual Private Cloud)
- Subnet
- Route Table
- Internet Gateway
- EC2 (Elastic Compute Cloud)

3. Architecture Overview

A custom VPC named MyCustomVPC was created with a CIDR block of 10.0.0.0/16. Inside this VPC, a public subnet (PublicSubnet) was created with a CIDR block of 10.0.1.0/24. An Internet Gateway (MyIGW) was attached to the VPC so that resources inside it could communicate with the internet. A route table (PublicRouteTable) was created and configured with a route sending all outbound traffic (0.0.0.0/0) to the Internet Gateway, and this route table was then associated with the public subnet. Finally, an EC2 instance was launched inside the public subnet with a public IP address to test internet connectivity.

VPC Name	MyCustomVPC
VPC CIDR	10.0.0.0/16
Subnet Name	PublicSubnet
Subnet CIDR	10.0.1.0/24
Internet Gateway	MyIGW
Route Table	PublicRouteTable
EC2 Instance	MyPublicEC2 (t3.micro)

4. Implementation Steps

Step 1: Creating the Custom VPC

The first step was to create a custom VPC named MyCustomVPC with a CIDR block of 10.0.0.0/16. The screenshot below shows the VPC listed alongside the default VPC in the AWS console, confirming that it was created successfully and is in the Available state.

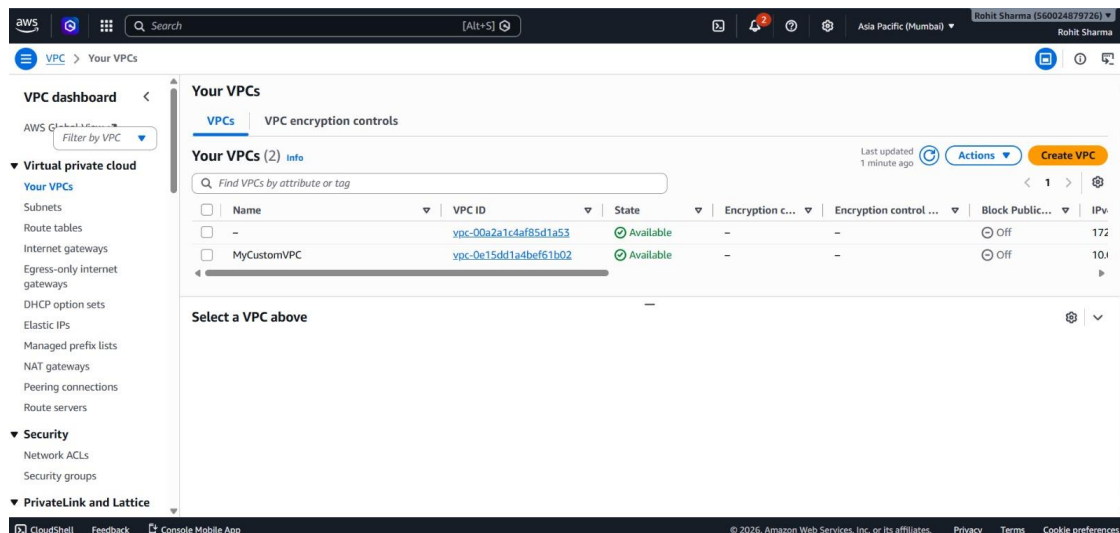


Fig 1: MyCustomVPC created and shown as Available

Step 2: Creating the Public Subnet

A subnet named PublicSubnet was created inside MyCustomVPC with a CIDR block of 10.0.1.0/24. This subnet will later be made 'public' by attaching it to a route table that has internet access.

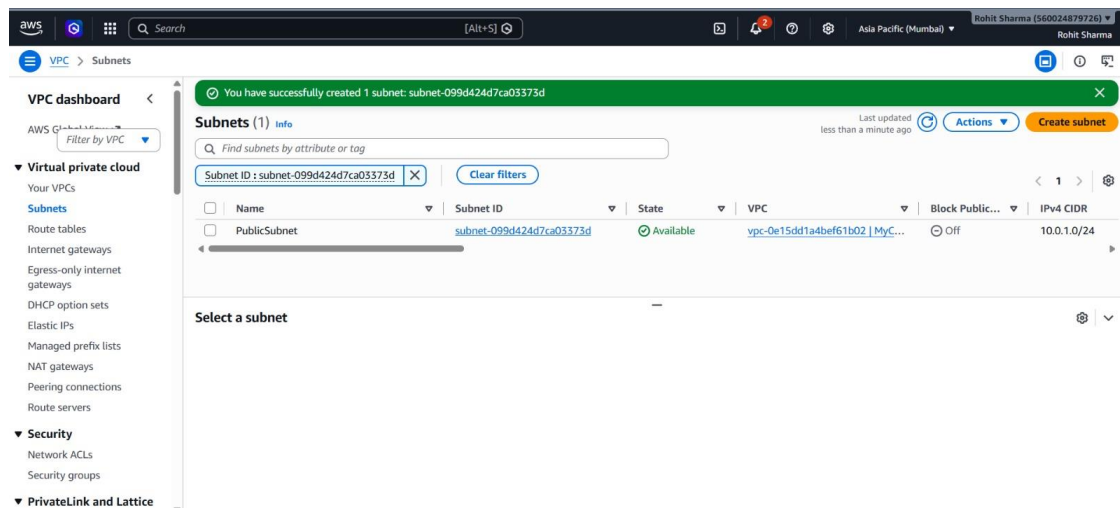


Fig 2: PublicSubnet created inside MyCustomVPC

Step 3: Creating the Internet Gateway

An Internet Gateway named MyIGW was created. At this point it was not yet attached to any VPC, so its state shows as Detached.

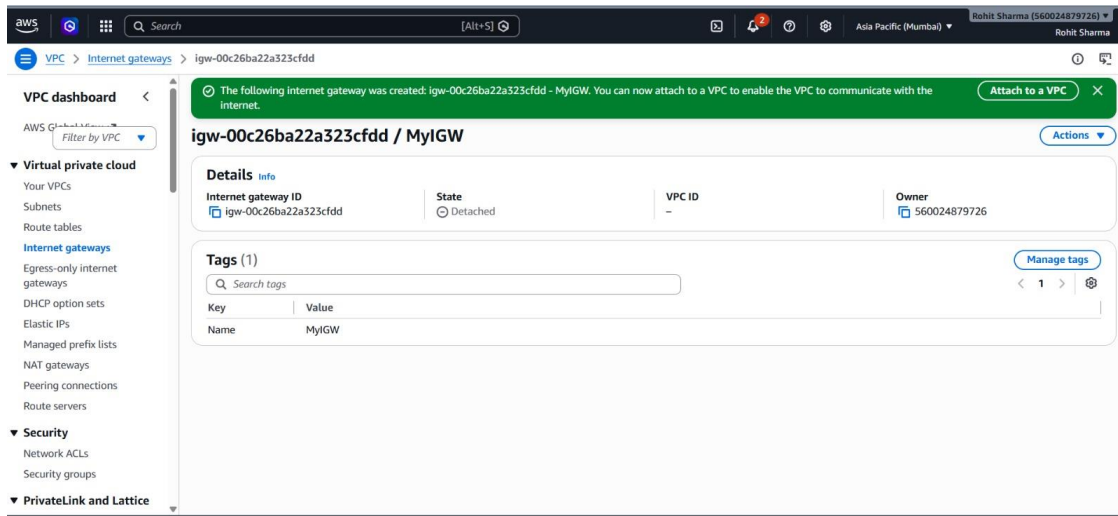


Fig 3: MyIGW created (initial state: Detached)

Step 4: Attaching the Internet Gateway to the VPC

The Internet Gateway was then attached to MyCustomVPC. Once attached, its state changed to Attached, meaning the VPC can now route traffic out to the internet through it.

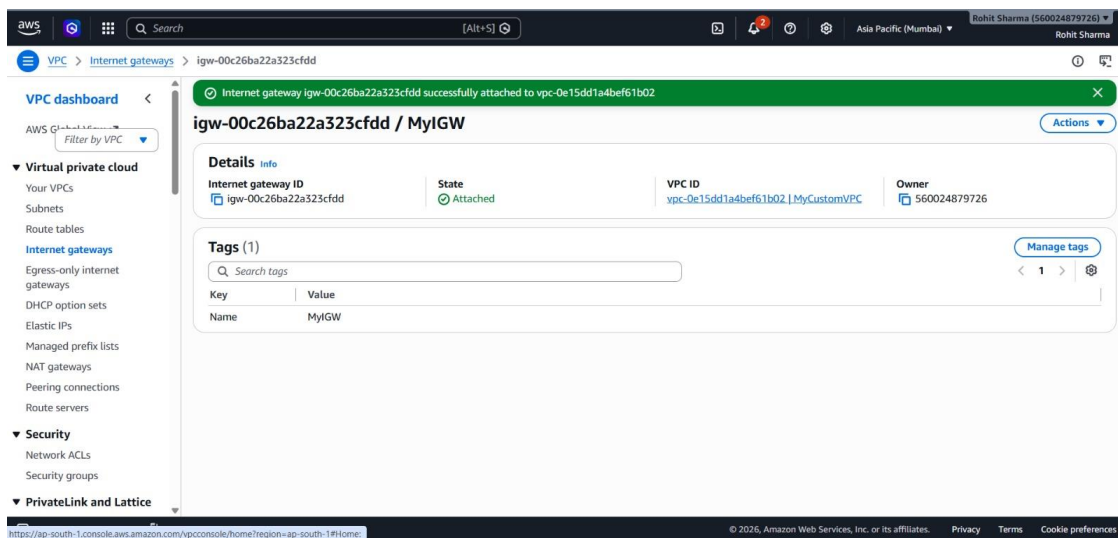


Fig 4: MyIGW successfully attached to MyCustomVPC

Step 5: Creating the Route Table

A route table named PublicRouteTable was created inside MyCustomVPC. By default, it only contains the local route (10.0.0.0/16), which allows communication within the VPC. A route to the internet still needed to be added.

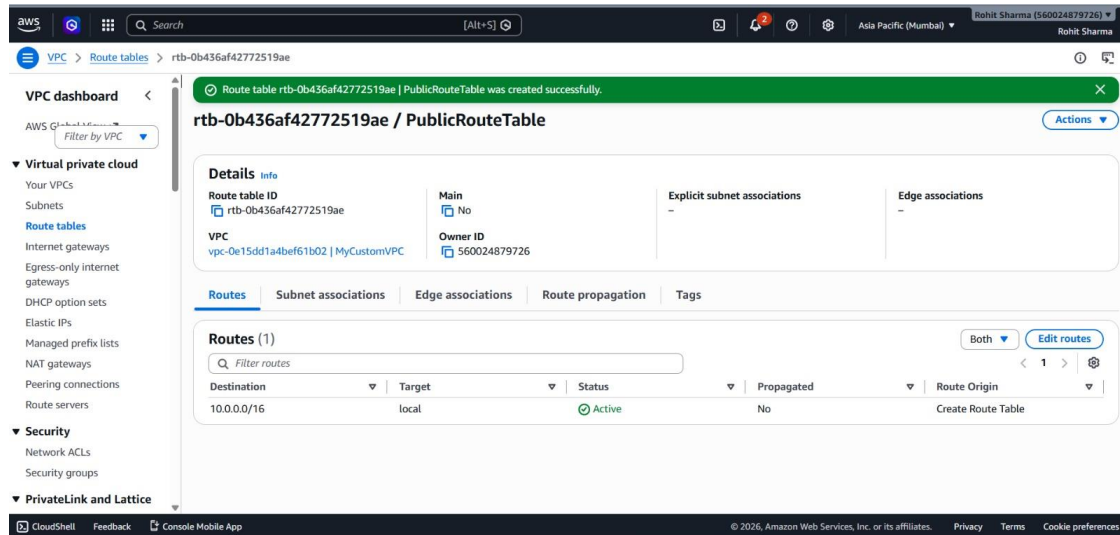


Fig 5: PublicRouteTable created with the default local route

Step 6: Adding the Internet Route and Associating the Subnet

A new route was added to PublicRouteTable, sending all traffic destined outside the VPC (0.0.0.0/0) to the Internet Gateway MyIGW. The route table was then associated with PublicSubnet, which is what actually makes the subnet public. The screenshot below confirms the subnet association was updated successfully.

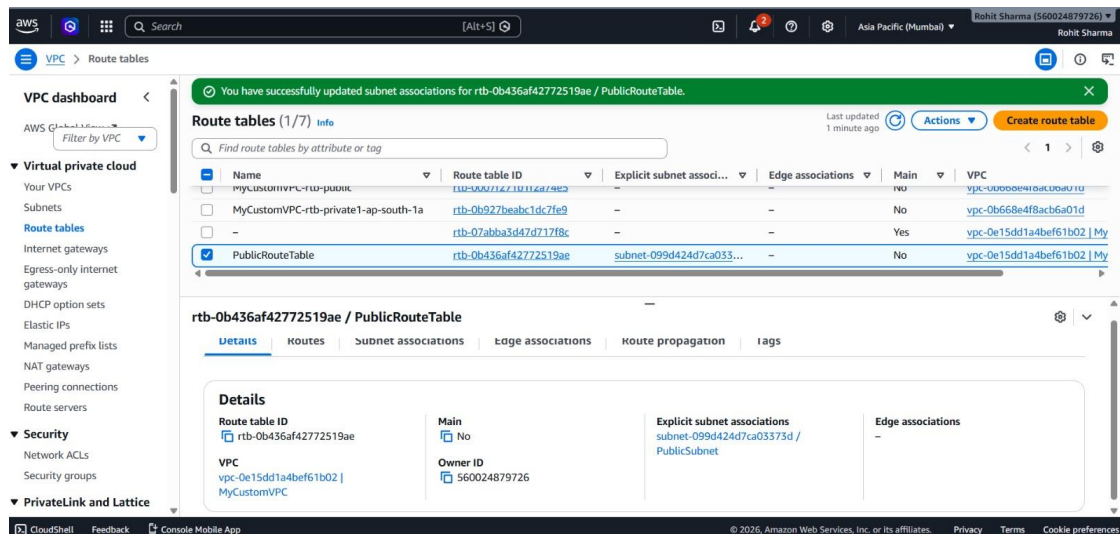


Fig 6: PublicRouteTable associated with PublicSubnet

Step 7: Launching the EC2 Instance

An EC2 instance named MyPublicEC2 (t3.micro) was launched inside PublicSubnet, with a public IP address automatically assigned. The instance reached the Running state and was assigned the public IPv4

address 15.207.85.241, confirming that it is reachable from the internet through the Internet Gateway and route table configured earlier.

The screenshot displays the AWS Management Console interface for an EC2 instance. The left sidebar shows the navigation menu with categories like EC2, Instances, Images, and Elastic Block Store. The main content area is titled 'Instance summary for i-0b8572e232d792d26 (MyPublicEC2)'. It provides a comprehensive overview of the instance's configuration and status.

Property	Value
Instance ID	i-0b8572e232d792d26
Public IPv4 address	15.207.85.241 open address
Instance state	Running
Private IPv4 addresses	10.0.1.55
Public DNS	-
Private IP DNS name (IPv4 only)	ip-10-0-1-55.ap-south-1.compute.internal
Instance type	t3.micro
Auto-assigned IP address	15.207.85.241 [Public IP]
VPC ID	vpc-0e15dd1a4bef61b02 (MyCustomVPC)
Subnet ID	subnet-099d424d7ca03373d (PublicSubnet)
Instance ARN	arn:aws:ec2:ap-south-1:560024879726:instance/i-0b8572e232d792d26
Hostname type	IP name: ip-10-0-1-55.ap-south-1.compute.internal
Answer private resource DNS name	-
Elastic IP addresses	-
AWS Compute Optimizer finding	Opt-in to AWS Compute Optimizer for recommendation Learn more
Auto Scaling Group name	-
Managed	false
IAM role	-
IMDSv2	Required

Fig 7: MyPublicEC2 instance running with a public IP address

5. Result

The custom VPC, public subnet, Internet Gateway, and route table were all configured correctly, and the EC2 instance launched inside the public subnet received a public IP address and reached the Running state. This confirms that the public subnet is correctly connected to the internet, and the objective of the project was achieved.

6. Conclusion

Through this project, we learned how the different networking components of AWS work together to create a public subnet. A subnet only becomes 'public' when it is associated with a route table that has a route pointing to an Internet Gateway. This hands-on exercise helped us understand the basics of VPC networking, which forms the foundation for more advanced multi-tier architectures involving private subnets, NAT gateways, and load balancers.

7. GitHub Repository

The project screenshots and this report have been uploaded to the following GitHub repository:

<https://github.com/rohitkr-dev45/AWS-VPC-Project>